

Banco Digital — Especificação Definitiva do Projeto BMVC

Arquitetura MVC · Controllers · Models · Views · Static · WebSocket · SQLite

Roteiro de construção passo a passo para desenvolvimento assistido por IA

Gabriel — Engenharia de Software (UnB)

Versão 2026.1

1. Visão Geral do Projeto

O BMVC é um projeto acadêmico incremental que simula um banco digital, construído em quatro níveis progressivos de complexidade. Cada nível corresponde a uma etapa de avaliação prática, na qual o estudante deve gravar um vídeo demonstrando o funcionamento real do código em sua própria máquina e disponibilizar o repositório de forma pública. O projeto segue o padrão arquitetural MVC (Model-View-Controller), com persistência em SQLite e, no nível mais avançado, comunicação em tempo real via WebSocket.

Este documento consolida a especificação completa, incluindo critérios de avaliação de cada nível e um roteiro sequencial de instruções que pode ser fornecido a uma IA assistente de programação para construir o projeto passo a passo, do zero até o Nível IV.

2. Arquitetura do Projeto

A estrutura de pastas segue o padrão MVC, separando claramente responsabilidades de dados, regras de negócio, apresentação e arquivos estáticos:

```
bmvc/
  controllers/      # lógica de rotas e regras de negócio
  models/          # classes de acesso e representação dos dados
  views/           # templates HTML/TPL
  static/
    css/
    js/
    img/
  websocket/       # handlers de eventos em tempo real (Nível IV)
  database/        # arquivo SQLite e scripts de migração
  app.py           # ponto de entrada da aplicação
  requirements.txt
```

3. Entidades e Modelo de Dados

3.1 Usuario

Campo	Tipo	Descrição
id	INTEGER (PK)	Identificador único, autoincremento
nome	TEXT	Nome completo do usuário
email	TEXT (UNIQUE)	E-mail usado como login
senha_hash	TEXT	Senha armazenada com hash (nunca em texto puro)
tipo	TEXT	Papel do usuário: 'cliente' ou 'admin'

3.2 Conta

Campo	Tipo	Descrição
id	INTEGER (PK)	Identificador único, autoincremento
numero_conta	TEXT (UNIQUE)	Número da conta bancária
usuario_id	INTEGER (FK)	Referência ao Usuario proprietário
saldo	REAL	Saldo atual da conta

Observação: Os campos nome e CPF pertencem a Usuario, e Conta se relaciona a Usuario por chave estrangeira (usuario_id), evitando duplicidade e seguindo normalização básica.

3.3 Transacao

Campo	Tipo	Descrição
id	INTEGER (PK)	Identificador único, autoincremento
conta_origem	INTEGER (FK)	Conta de origem (NULL em depósitos)
conta_destino	INTEGER (FK)	Conta de destino (NULL em saques)
tipo	TEXT	'deposito', 'saque' ou 'transferencia'
valor	REAL	Valor movimentado
data_hora	DATETIME	Data e hora do registro da transação
descricao	TEXT	Descrição opcional da movimentação

4. Níveis do Projeto e Critérios de Avaliação

Cada nível combina o escopo técnico com os critérios de avaliação. Todos os níveis exigem página(s) com propósito claro, CSS e JS próprios (nunca reaproveitados do material de sala) e funcionamento demonstrado ao vivo em vídeo.

Nível I — Landing Page Institucional

Escopo técnico:

- Páginas: Home, Sobre, Serviços e Contato, servidas como HTML/TPL estático pelo Controller
- CSS personalizado: layout responsivo, navbar, cards e rodapé
- JavaScript: menu responsivo, modo escuro (dark mode), scroll suave e botão 'voltar ao topo'
- Objetivo funcional: servir corretamente páginas estáticas via servidor Python

Critérios de avaliação (BMVC I):

Vídeo (YouTube): Até 3 minutos

Deve demonstrar: Funcionamento completo do código servindo uma página estática (HTML/TPL), com propósito claro

Não é aceito: Reutilizar o mesmo HTML apresentado em sala — o código deve ser personalizado

Entrega obrigatória: Link do vídeo + link do repositório público (GitHub)

Nível II — CRUD de Contas e Transações

Escopo técnico:

- CRUD completo de Contas (id, usuario_id, número da conta, saldo)
- CRUD completo de Transações (depósito, saque e transferência)
- Páginas para criar, editar, listar e excluir registros de ambas as entidades
- Persistência em banco SQLite, acessado pelos Models

Critérios de avaliação (BMVC II):

Vídeo (YouTube): Até 6 minutos

Deve demonstrar: Funcionamento completo do CRUD de um ou mais modelos Python em uma ou mais páginas HTML

Não é aceito: Reutilizar o mesmo modelo ou a mesma página HTML apresentados em sala

Entrega obrigatória: Link do vídeo + link do repositório público (GitHub)

Nível III — Autenticação e Dashboard

Escopo técnico:

- Cadastro de usuários, login e logout, com senha protegida por hash
- Páginas protegidas: acesso restrito a usuários autenticados (controle de sessão)
- Dashboard exibindo saldo atual, últimas movimentações e dados de perfil do usuário logado

Critérios de avaliação (BMVC III):

Vídeo (YouTube): Até 10 minutos

Deve demonstrar: CRUD completo de um ou mais modelos + sistema de login funcional e página de acesso restrito com controle de sessão

Não é aceito: Reutilizar os mesmos modelos ou páginas HTML apresentados em sala

Entrega obrigatória: Link do vídeo + link do repositório público (GitHub)

Nível IV — Tempo Real com WebSocket

Escopo técnico:

- Canal WebSocket dedicado para atualização assíncrona de saldo, histórico e painel administrativo
- Atualizações refletidas na página sem necessidade de recarregar (F5)

- Painel administrativo mostrando eventos de transações em tempo real

Critérios de avaliação (BMVC IV):

Vídeo (YouTube): Até 15 minutos

Deve demonstrar: CRUD completo + login/página restrita + comunicação WebSocket com atualização assíncrona de dados em tempo real

Não é aceito: Reutilizar os mesmos modelos ou páginas HTML apresentados em sala

Entrega obrigatória: Link do vídeo + link do repositório público (GitHub)

5. Roteiro de Construção Passo a Passo para uma IA

As instruções abaixo podem ser fornecidas, em sequência, a uma IA de programação (ex.: Claude Code) para construir o projeto do zero até o Nível IV. Cada etapa deve ser concluída, testada e validada antes de avançar para a próxima.

Etapas 0 — Preparação do Ambiente

- Criar repositório público no GitHub chamado 'bmvc-banco-digital'
- Criar estrutura de pastas: controllers/, models/, views/, static/css/, static/js/, static/img/, database/
- Configurar ambiente virtual Python (venv) e um requirements.txt inicial (framework web leve, ex.: Flask)
- Criar app.py como ponto de entrada, com rota inicial de teste
- Criar README.md descrevendo o propósito do projeto (banco digital fictício)

Etapas 1 — Nível I: Landing Page

- Criar as views Home, Sobre, Serviços e Contato em views/, com conteúdo textual original e coerente com um banco digital fictício
- Criar static/css/style.css com layout responsivo (media queries), navbar fixa, cards de serviços e rodapé
- Criar static/js/main.js implementando: toggle de menu responsivo, alternância de modo escuro (persistindo preferência), scroll suave entre seções e botão flutuante 'voltar ao topo'
- Configurar as rotas no Controller para servir cada página estática
- Testar localmente todas as páginas e interações de JS antes de gravar o vídeo do Nível I

Etapas 2 — Nível II: CRUD de Contas e Transações

- Criar o banco SQLite em database/bmvc.db com as tabelas Usuario, Conta e Transacao
- Implementar em models/ as classes Conta e Transacao, com métodos de criar, listar, atualizar e excluir (parametrizando queries para evitar SQL Injection)
- Implementar em controllers/ as rotas de CRUD para Conta: listar, criar, editar e excluir
- Implementar em controllers/ as rotas de CRUD para Transacao, incluindo as três operações: depósito, saque e transferência, atualizando o saldo da(s) conta(s) envolvida(s)
- Criar as views correspondentes (formulários e listagens), com CSS e JS próprios
- Validar regras básicas: saldo não pode ficar negativo em saques/transferências
- Testar todo o fluxo de CRUD localmente antes de gravar o vídeo do Nível II

Etapas 3 — Nível III: Autenticação e Dashboard

- Implementar em models/ a classe Usuario, com criação de conta e armazenamento de senha em hash (ex.: bcrypt ou werkzeug.security)
- Implementar rotas de cadastro, login e logout no Controller, com gerenciamento de sessão
- Criar um decorator/middleware de proteção de rotas, redirecionando usuários não autenticados para a página de login
- Criar a view de Dashboard, exibindo: saldo da conta do usuário logado, lista das últimas movimentações e dados de perfil
- Vincular cada Conta a um Usuario (usuario_id), de modo que o Dashboard mostre apenas dados do usuário autenticado
- Testar login, logout, proteção de rotas e conteúdo do dashboard antes de gravar o vídeo do Nível III

Etapas 4 — Nível IV: Tempo Real com WebSocket

- Adicionar dependência de WebSocket ao projeto (ex.: Flask-SocketIO ou biblioteca equivalente)
- Criar em websocket/ os handlers de eventos: atualização de saldo, nova transação e evento para o painel administrativo

- No Controller de Transacao, emitir um evento WebSocket sempre que uma transação for concluída com sucesso
- No JS do Dashboard, escutar os eventos e atualizar dinamicamente saldo e histórico, sem recarregar a página
- Criar (ou estender) um painel administrativo que escute eventos de todas as transações do sistema em tempo real
- Testar o fluxo completo com duas abas abertas simultaneamente (dois usuários), verificando que as atualizações aparecem em tempo real, antes de gravar o vídeo do Nível IV

6. Checklist Final de Entrega

Para cada nível concluído, confirme os itens abaixo antes de enviar a atividade:

- Repositório GitHub público, com código do nível correspondente commitado
- Vídeo gravado e publicado no YouTube, respeitando o tempo máximo do nível
- Vídeo demonstra o código rodando localmente ('na máquina do estudante'), não apenas o código-fonte
- Página(s) com propósito claro e conteúdo legível, sem reaproveitar material apresentado em sala
- CSS e JS carregando corretamente e plenamente funcionais durante a demonstração
- Link do vídeo e link do repositório entregues juntos, na atividade correspondente ao nível

Este documento substitui o rascunho inicial e serve como especificação única do projeto BMVC, orientando tanto o desenvolvimento técnico quanto a preparação dos vídeos de avaliação de cada nível.